index

Numerals

127.0.0.1 IP address 331

200 OK response status 316

Symbols

* operator 167

A

@Async annotation 116, 118

AbstractAuthenticationToken class 308

access

restricting based on authorities and roles 139–148

restricting based on user roles 148–153

restricting to all endpoints 153

Access-Control-Allow-Headers header 201

Access-Control-Allow-Methods header 201, 395

Access-Control-Allow-Origin header 201, 202, 204, 395

AccessDeniedException 125, 212, 388

access() method 141, 145, 146, 147, 150, 152, 361, 362, 363

addFilterAfter() method 93

addFilterAt() method 96

addFilterBefore() method 90

allowedHeaders attribute 206

anyExchange() method 361

anyRequest() method 31, 143, 156, 160, 361

AOP (aspect-oriented programming) 211

applyDefaultSecurity() method 278

asymmetric keys 78

authenticate(Authentication authentication) method 36

authenticated() method 361

authenticate() method 106, 107, 108, 109

authentication 101

AuthenticationProvider 103–112

applying custom authentication logic 108–112

implementing custom authentication logic 105–112

representing request during authentication 104

form-based login 123–133

implementing 127–133

HTTP Basic 123–133

configuring 124–126

implementing basic using JSON web tokens 276–284

SecurityContext 112–123

forwarding with DelegatingSecurityContextExecutorService 121–123

forwarding with DelegatingSecurityContextRunnable 119–121

holding strategy for asynchronous calls 116

holding strategy for standalone applications 118

holding strategy for security context 114

testing 390–394

AuthenticationEntryPoint 125

AuthenticationException 107, 109, 110, 125, 388

AuthenticationFailureHandler 391

implementation 131

AuthenticationFailureHandler object 130

authentication filter 90

Authentication instance 112, 351, 363

Authentication interface 38, 103

AuthenticationLoggingFilter class 99

AuthenticationManager 45, 102, 107, 112

authenticationManagerResolver() method 319, 322

Authentication object 112, 114, 226, 245, 336, 362, 376

AuthenticationProvider 23, 34, 35, 38, 45, 69, 103–112, 107, 109–111, 385, 390

implementing custom authentication logic 105–112

representing request during authentication 104

AuthenticationProvider contract 108

AuthenticationSuccessHandler object 130

AuthenticationWebFilter 355

authorities 27

restricting access based on 139–148

authorities() method 152, 394

authorization

configuring rules in reactive apps 358–366

applying authorization at endpoint layer 358–364

using method security 365

defined 137

endpoint-level

applying restrictions 156

selecting requests to apply authorization restrictions 162–169

using requestMatchers() method to select endpoints 157–162

implementing at method level 223–232

authorization code grant type 266–268

applying PKCE protection to 269

running 284–290

AuthorizationContext 362

AuthorizationDecision 363

Authorization header 19, 20, 23, 94, 95, 98, 305

AuthorizationManager contract 141, 152

AuthorizationManager implementation 145, 147

Authorization object 117

Authorization request header 337

authorization server 324

role of 275, 276

using opaque tokens and introspection 292–294

AuthorizationServerSettings 277, 284

AuthorizationWebFilter 363, 364

authorizeExchange() method 360

authorizeHttpRequests() method 31, 32, 143, 360

availability zone (AZ) 10

B

@Bean annotation 27

@Bean method 252

BasicAuthenticationFilter 86

BasicAuthenticationFilter class 90, 96, 97

BCryptPasswordEncoder 72, 74, 75, 76, 77

Bearer prefix 305

body 265

BookService class 221

build() method 52, 361

BytesEncryptor interface 80, 81

BytesKeyGenerator interface 79, 80

C

@Component annotation 108, 111

@ComponentScan annotation 18

@Configuration annotation 27, 116

@Controller 129

@Controller class 203

@CrossOrigin annotation 205, 207

_csrf attribute 179, 180, 186

Callable interface 123

Callable object 120

Callable task 120

Callable<T> alternative 119

call authorization 211–213

postauthorization 213

preauthorization 212

CBC (cipher block chaining) 81

chains

adding filters at location of another in 93–98

adding filters before existing one in 88–91

challenge 269

client authentication meeting 332

client credentials grant type 270, 324

running 290

ClientRegistration object 333

ClientRegistrationRepository component 334, 339

ClientRegistrationRepository contract 333

ClientSettings instance 287

clientSettings() method 287

Cloud Native Spring in Action (Vitale) 9

commence() method 125

CommonOAuth2Provider class 328

configuration classes 315, 365, 367

using multiple 38

configuring

in different ways 32

token validation through introspection 311–316

contracts 26

controllers package 18

convention-over-configuration 15

Converter 309

CorsConfiguration object 207

CorsConfigurationSource 207

CorsConfigurer 207

CORS (Cross-Origin Resource Sharing) 277, 376

applying policies with @CrossOrigin annotation 205

configuring 199

overview of 200–205

testing configurations 395

CorsFilter 86

cors() method 207

createDelegatingPasswordEncoder() method 77

createSecurityContext() method 385

CSRF (cross-site request forgery) 90, 175, 176, 273, 277, 376

customizing protection 188–197

overview 176–182

protection, using in practical scenarios 182–188

testing configurations 394

CsrfCustomizer object 190

CsrfFilter 86, 177, 179

csrf() method 189

csrf() RequestPostProcessor 394

CsrfToken 180, 191

CsrfToken contract 190, 191

CsrfToken interface 191

CsrfTokenLogger 179, 180

CsrfToken object 190

CsrfTokenRepository 178, 179, 181, 182, 195, 196

implementation 193

interface 191

csrfTokenRepository() method 195

CsrfTokenRequestAttributeHandler 196

CsrfTokenRequestHandler 196

CsrfToken type 191

curl calls 98

custom Authentication objects, using for testing 384–388

step 1 (defining custom annotation) 386

step 2 (creating factory class for mock SecurityContext) 387

step 3 (linking custom annotation to factory class) 387

CustomCsrfTokenRepository 193, 194

CustomEntryPoint 126

customized JWTs 306–311

Customizer 207

Customizer interface 31, 302

Customizer<CorsConfigurer> object 207

Customizer<HttpBasicConfigurer<HttpSecurity>> parameter 125

Customizer object 31, 188, 189, 302, 367

Customizer type 124

customizing user details management 26–29

D

@DisplayName annotation 380

data at rest 11

data in transition 11

data.sql file 250

DDoS (distributed denial of service) 4

DefaultCsrfToken 191

DefaultMethodSecurityExpressionHandler 227

defaultSuccessUrl() method 130

defining custom authentication logic 34–38

DelegatingPasswordEncoder 74–77

DelegatingSecurityContextCallable 120–123

DelegatingSecurityContextCallable<T> 119

DelegatingSecurityContextExecutor 122, 123

DelegatingSecurityContextExecutorService 121–123

DelegatingSecurityContextRunnable 119, 121, 123

DelegatingSecurityContextScheduledExecutorService 122, 123

denyAll() method 153, 154, 161, 167

Document class 224

DocumentRepository class 224

DocumentService class 224, 230

DocumentsPermissionEvaluator class 226

doFilterInternal() method 99

doFilter() method 86, 89, 98

DummyUser class 49, 50

E

@EnableAsync annotation 116

@EnableGlobalMethodSecurity annotation 214

@EnableMethodSecurity annotation 213, 214, 232, 238

@EnableReactiveMethodSecurity annotation 365

Employee class 221

encode(CharSequence rawPassword) method 70

encode() method 70, 71

encryption 77

encryptors 80

Encryptors.delux() method 81

Encryptors factory class 81

Encryptors.noOpText() method 81

Encryptors.standard() method 81

Encryptors.stronger() method 81

Encryptors.text() method 81

endpoint-level authorization

applying restrictions 156

configuring 137

restricting access based on authorities and roles 139–148

restricting access based on user roles 148–153

restricting access to all endpoints 153

selecting requests to apply authorization restrictions 162–169

using regular expressions with request matchers 169–173

using requestMatchers() method to select endpoints 157–162

endpoints, applying authorization at endpoint level 29–32

equals() method 71

equals(Object o) method 28

ExceptionTranslationManager 125

exclude attribute 98

Executor interface 123

Executor object 123

ExecutorService interface 123

ExecutorService object 123

F

/find endpoint 246

filter chain 86

FilterChain object 86

FilterChain parameter 86

filter chains 86

adding filter after existing one in chain 91

filter implementations, provided by Spring Security 98–100

Filter interface 86, 98, 99, 100

filterObject 239, 245

filters

adding after existing one in chain 91

adding at location of another in chain 93–98

adding before existing one in chain 88–91

implementing in Spring Security architecture 85–87

findAll() method 247

findProducts() method 245

findTokenByIdentifier() method 193

form-based login authentication 123–133

implementing 127–133

FormLoginConfigurer<HttpSecurity> object 130, 132

formLogin() method 127, 128, 130, 132, 133, 183, 302, 336

G

@GetMapping 240

@GetMapping annotation 19, 162

GCM (Galois/Counter Mode) 81

generateKey() method 79, 80

generateToken() method 194, 196

GenericFilterBean class 98, 99

getAuthorities() method 47, 50, 54, 140

getAuthority() method 49, 54, 139

getContext() method 114

getName() method 215, 232, 388

getPassword() method 47, 50

getRequestPath() method 363

getSecretNames() method 218

getToken() method 180

getUsername() method 47, 50

GrantedAuthority 44

GrantedAuthority contract 48, 139, 150

GrantedAuthority instances 140

GrantedAuthority interface 46, 139

GrantedAuthority object 151

grant types

applying PKCE protection to authorization code grant type 269

authorization code grant type 266–268

client credentials grant type 270

obtaining tokens through various 266–271

refresh tokens 271

H

/hello endpoint 197

hasAnyAuthority() method 141, 144, 145, 146, 217, 361

hasAnyAuthority('READ', 'WRITE') method 146

hasAnyRole() method 150, 217, 361

hasAuthority() method 140, 141, 144, 145, 146, 216, 361

hashing 77, 78

hasPermission() method 224, 226

hasRole() method 150, 217, 361

header 264

Heckler, Mark 17, 401

HelloController class 18, 189, 350, 351, 356

HelloController method 19

Hello! message 351

High-Performance Java Persistence (Mihalcea) 401

holding strategy

for asynchronous calls 116

for SecurityContext 114

for standalone applications 118

HTTP Basic authentication 123–133

configuring 124–126

HttpBasicConfigurer<HttpSecurity> parameter 125

httpBasic() method 31, 86, 97, 124, 127, 302, 336, 367

httpBasic() request postprocessor 391

HTTP OPTIONS method 395

HttpSecurity class 86, 97

HttpSecurity instance 124

HttpSecurity object 30, 90, 127, 189, 207, 302

HttpServletRequest 100, 125

HttpServletResponse 100, 125

I

if-else clause 36

ignoringRequestMatchers() method 190

ignoringRequestMatchers(String paths) method 190

implicit grant type 268

InMemoryUserDetailsManager 26, 44, 142, 157, 164, 354

interface segregation principle 46

introspection 293, 292–294

configuring token validation through 311–316

isAccountNonExpired() method 48

isAccountNonLocked() method 48

isAdmin() method 363

isAuthenticated() method 106

isCredentialsNonExpired() method 48

isEnabled() method 48

J

Java Persistence with Spring Data and Hibernate (Tudose) 402

JdbcUserDetailsManager 44

JKS (Java KeyStore) format 24

JpaRepository class 192

JpaRepository contract 193

JpaTokenRepository 193

JpaTokenRepository interface 193

JSON (JavaScript Object Notation) 265

JSON web tokens (JWTs), implementing basic authentication using 276–284

jsr250Enabled attribute 232

JUnit in Action (Tudose) 402

jwkSetUri() method 302

JWKSource 283

JwtAuthenticationProvider 322

JwtIssuerAuthenticationManagerResolver 318

JwtIssuerAuthenticationResolver 319

jwt() method 302, 367, 393

JWTs (JSON web tokens) 298

configuring validation 298–306

using customized 306–311

jwtSpec instance 368

K

-k option 24

key1 80

key2 80

key generators 79

KeyGenerators factory class 79

KeyGenerators.secureRandom() method 80

KeyGenerators.shared(int length) method 80

key pair 78

L

LdapUserDetailsManager 44

List.of() method 241

LIVENESS scope 290

loadToken() method 195

loadUserByUsername() method 109

localhost DNS name 331

M

MainController class 184

MainTests class 378

MapReactiveUserDetailsService 354

matches(CharSequence rawPassword, String encodedPassword) method 70

matches() method 70, 71, 109

method authorization, applying postfiltering for 242–247

method-level authorization

applying preauthorization rules 214–219

postauthorization rules 219–223

method-level filtering 235

prefiltering 236–242

Spring Data repositories 247–253

methods, implementing permissions for 223–233

methods attribute 206

method security 209

enabling 210–214

MethodSecurityExpressionHandler bean 226, 227

Mihalcea, Vlad 401

MockMvc 389

mock users, using for tests 376–383

MODE_GLOBAL mode 119

MODE_INHERITABLETHREADLOCAL strategy 117, 118

MODE_INHERITEDTHREADLOCAL mode 119

MODE_THREADLOCAL mode 119

Mono<Authentication> 356

Mono return type 350

multitenant systems 317–322

Musib, Somnath 17

MvcRequestMatcher 190

my_authorization_server 330, 331

N

NameService 389

NameService class 218, 388

non-opaque tokens 263, 264

NoOpPasswordEncoder 28, 70, 72, 73, 75, 76, 77, 280

NoOpPasswordEncoder.getInstance() method 73

O

@Order annotation 279

OAuth 2 188, 257–258

creating reactive resource server 367–369

implementing clients 336–342

obtaining tokens through various grant types 266–271

applying PKCE protection to authorization code grant type 269

authorization code grant type 266–268

client credentials grant type 270

refresh tokens 271

OpenID Connect, overview of 272

overview of 259–262

resource servers

configuring token validation through introspection 311–316

implementing 297

using customized JWTs 306–311

security vulnerabilities 273

tokens 262–265

non-opaque tokens 264

opaque tokens 263

OAuth2AuthenticationPrincipal class 336

OAuth 2 authorization server 275

implementing basic authentication using JSON web tokens 276–284

revoking tokens 295

running authorization code grant type 284–290

running client credentials grant type 290

using opaque tokens and introspection 292–294

OAuth2AuthorizationServerConfigurer 278

OAuth2AuthorizedClientManager class 340

OAuth 2 client 324

implementing OAuth 2 login 326

OAuth 2 client dependency 326

oauth2Client() method 339

OAuth 2 login

adding flexibility to configurations 333

giving user more possibilities 329

implementing 326–336

implementing authentication with common provider 326–328

managing authorization for OAuth 2 login 335

using custom authorization server 330–332

oauth2Login() method 335

OAuth 2 resource servers, configuring JWT validation 298–306

oauth2ResourceServer() method 302, 314, 336, 367

OAuth2TokenCustomizer bean 307

official documentation, links to 399

oidc() method 278

onAuthenticationFailure() method 131

onAuthenticationSuccess() method 130

OncePerRequestFilter 98

OncePerRequestFilter class 99, 100

OpaqueTokenAuthenticationProvider 322

opaqueToken() method 314, 394

opaque tokens 263, 292–294

OpenID Connect 259–262, 272

openssl command 24

OTP (one-time password) 93

owner attribute 239

P

@PostAuthorize annotation 220, 224, 232, 234, 235, 238

@PostFilter annotation 239, 243, 245, 247, 251

@PostMapping annotation 162

@PreAuthorize 388

@PreAuthorize annotation 216, 230, 232, 234, 235, 238, 365

@PreFilter annotation 237, 238, 239, 240, 247

PasswordEncoder 27, 44, 45, 68, 77, 101, 215, 227, 277, 279, 354–356, 359

PasswordEncoder contract 70

PasswordEncoderFactories class 77

PasswordEncoder interface 52

password encoders 68–78

choosing from provided implementations 72–74

DelegatingPasswordEncoder 74–77

implementing 70

passwords

password encoders 68–78

SSCM (Spring Security Crypto module) 78–81

encryptors 80

key generators 79

pathMatchers() method 360

Pbkdf2PasswordEncoder 72, 73

PermissionEvaluator 226, 234

PermissionEvaluator contract 225, 226

PermissionEvaluator implementation 226

PermissionEvaluator method 230

permissions, implementing for methods 223–233

permitAll() method 30, 31, 90, 143, 144, 153, 160, 161

PKCE (proof key for code exchange) 269

PKCS12 (Public Key Cryptography Standards #12) 24

PlainTextPasswordEncoder 73

postauthorization 213

rules 219–223

postfiltering 235

applying for method authorization 242–247

preauthorization 212

rules 214–219

prefiltering 235, 236–242

preflight request 205

principal 104

private key 78

Product class 239

ProductController class 185, 240, 246, 251

Product entity 250, 251

Product entity class 250

ProductRepository interface 251, 252

ProductService class 239, 240, 245, 246

ProjectConfig class 214, 218, 354

provider name 332

public key 78

publicKey() method 368

Publisher contract 350

Publisher implementation 350

Publisher interface 350

Publisher object 354

R

@RequestMapping annotation 162

@ResponseBody annotation 203

@RestController 129

@RestController annotation 19

@Retention (RetentionPolicy.RUNTIME) annotation 386

@RolesAllowed annotation 214, 232, 233, 234

reactive applications, creating reactive OAuth 2 resource server 367–369

reactive apps

configuring authorization rules in 358–366

applying authorization at endpoint layer 358–364

using method security 365

overview of 348–352

user management in 353–358

ReactiveAuthenticationManager 355, 356

ReactiveAuthorizationManager 363, 364

ReactiveSecurityContextHolder 356, 357

reactive Spring Security, testing implementations 396

ReactiveUserDetailsService 353, 354, 355–356, 359

realmName() method 125

redirect URI 332

redirectUri() method 282

refresh tokens 271

RegexRequestMatcher 170

RegisteredClient 280, 281, 282

RegisteredClientRepository 277, 280, 281, 282

RegisteredClientRepository definition 312

regression testing 374

regular expressions, using with request matchers 169–173

Request-Id header 88, 91

RequestMatcher 190

request matchers, using regular expressions with 169–173

requestMatchers() method 156, 157–162, 169, 170, 174, 360

RequestPostProcessor 381, 394

RequestValidationFilter class 88, 90

resource servers

configuring JWT validation 298–306

configuring token validation through introspection 311–316

implementing 297

implementing multitenant systems 317–322

using customized JWTs 306–311

returnObject method 221

reverse function decryption 77

ROLE_ADMIN attribute 363

role() parameter 152

ROLE_ prefix 150, 151, 152

roles, restricting access based on 148–153

roles() method 151, 152

Runnable interface 123

S

@Secure annotation 232

@Secured annotation 214, 232, 233, 234

/sell endpoint 242

@SpringBootApplication annotation 18, 98, 116

@SpringBootTest annotation 378

-supports() method 106

salt 78

saveToken() method 194

ScheduledExecutorService 122

ScheduledExecutorService interface 123

ScheduledExecutorService object 123

schema.sql file 250

SCryptPasswordEncoder 72, 74–77

securedEnabled attribute 232

SecureRandom object 74, 196

security 3

importance of 12–13

software security 8–11

testing configurations, with users from UserDetailsService 383

SecurityConfig class 307

SecurityContext 102, 112–123, 239, 245, 356, 358, 384, 385, 388, 389, 397

forwarding with DelegatingSecurityContextExecutorService 121–123

forwarding with DelegatingSecurityContextRunnable 119–121

holding strategy 114

for asynchronous calls 116

for standalone applications 118

SecurityContextHolder 113, 119

SecurityContextHolder class 115

SecurityContextHolder.setStrategyName() method 117

SecurityContext implementation 351

SecurityContext interface 103, 113

SecurityEvaluationContextExtension 252

SecurityFilterChain bean 29, 32, 88, 110, 127, 142, 207, 301

securityFilterChain() method 132, 150, 163, 189

SecurityUser class 54, 55

SecurityWebFilterChain bean 364

SecurityWebFilterChain contract 358

SecurityWebFilterChain object 361

SecurityWebFiltersOrder enum 87

sellProducts() method 239

ServletRequest object 86

ServletRequest parameter 86

ServletResponse object 86

ServletResponse parameter 86

setStrategyName() method 117

shouldNotFilterAsyncDispatch() method 100

shouldNotFilterErrorDispatch() method 100

shouldNotFilter(HttpServletRequest) method 100

signature 265

SimpleGrantedAuthority class 49

software security 8–11

SpEL (Spring Expression Language) 7, 216, 239

spring-boot-starter-data-jpa dependency 249

spring-boot-starter-security dependency 18, 141

spring-boot-starter-web dependency 18, 141

Spring Boot Up & Running (Heckler) 401

Spring Data repositories 247–253

Spring Expression Language (SpEL) 239

Spring in Action (Walls) 400

Spring in Practice (Wheeler and White) 16

Spring Security 15

architecture 21–25

alternatives to 7

authentication in 44–46

filter implementations provided by 98–100

filters, implementing in Spring Security architecture 85–87

overriding default configurations 25–39

applying authorization at endpoint level 29–32

customizing user details management 26–29

configuring in different ways 32

defining custom authentication logic 34–38

using multiple configuration classes 38

overview of 5–7

starting first project 16–20

Spring Start Here (Spilcă) 401

SSCM (Spring Security Crypto module) 68, 78–81

encryptors 80

key generators 79

StandardPasswordEncoder 72, 73

StaticKeyAuthenticationFilter class 95

String class 28

StringKeyGenerator interface 79

String value 190

Subscriber contract 350

supports(-Class<?> authentication) method 106

supports(Class<?> c) method 108

supports() method 36, 107, 108

symmetric key 77

T

@Transactional annotation 213

testing

authentication 390–394

CORS configurations 395

CSRF configurations 394

method security 388

reactive Spring Security implementations 396

with users from UserDetailsService 383

testing security configurations 373

using custom Authentication objects for testing 384–388

using mock users for tests 376–383

TextEncryptors 80, 81

ThreadLocal 356

tokens 190, 262–265

non-opaque tokens 264

obtaining through various grant types 266–271

applying PKCE protection to authorization code grant type 269

authorization code grant type 266– 268

client credentials grant type 270

refresh tokens 271

opaque tokens 263

revoking 295

token validation, configuring through introspection 311–316

Tudose, Cătălin 402

U

-u flag 20

-v flag 125, 181

upgradeEncoding(CharSequence encodedPassword) method 70

User builder class 27, 51, 151

User class 28

UserDetails 44, 46, 280

UserDetails contract 47–51, 101, 139, 140

UserDetails interface 68, 139

UserDetailsManager 44, 46

UserDetailsManager contract 101

UserDetailsService 44–46, 68, 98, 109, 110, 142, 157, 164, 215, 227, 279, 280, 376

customizing user details management 26–29

testing with users from 383

UserDetailsService bean 26, 277, 384

UserDetailsService component 351

UserDetailsService contract 101

UserDetails type 51

user management, in reactive apps 353–358

UserManagementConfig class 38, 39

user() method 381

UsernamePasswordAuthenticationToken class 108

users

authentication in Spring Security 44–46

describing 47–55

combining multiple responsibilities related to user 53–55

using builder to create instances of UserDetails type 51

with GrantedAuthority contract 48

with UserDetails contract 47

writing minimal implementation of UserDetails 49

User.withUsername(String username) method 52

UUID (universally unique identifier) 22, 194

V

var 27

varargs parameter 144

verifier 269

Vitale, Thomas 9

VM (virtual machine) 10

W

@WithCustomUser annotation 386, 387

@WithMockUser 388, 389

@WithMockUser annotation 380

@WithSecurityContext annotation 385, 387

@WithUserDetails 388, 389

@WithUserDetails annotation 384

Walls, Craig 400

web app security, filter implementations provided by Spring Security 98–100

web app's requested scope 332

WebAuthorizationConfig class 38, 39

WebExpressionAuthorizationManager 141, 150, 152

WebSecurityConfigurerAdapter class 32

WebTestClientConfigurer 396, 397

Wheeler, Willie 16

White, Joshua 16

withDefaults() method 31

with(httpBasic()) method 393

with(jwt()) method 393

with() method 381

with(opaqueToken()) method 393

WithSecurityContextFactory interface 385

WWW-Authenticate header 125

Previous chapter

< Appendix B. Further reading